UNIVERSIDADE ESTADUAL DE MARINGÁ
INFORMÁTICA - GRADUAÇÃO

GUSTAVO F. N. BONASSOLI	125235
INGRID LOUISE PEREIRA LOHMANN	117698
LARISSA EMY KASSUYA	102452

TRABALHO FINAL -
GERENCIAMENTO DE PROJETOS DE SOFTWARE

MARINGÁ

2025

GUSTAVO F. N. BONASSOLI 125235
INGRID LOUISE PEREIRA LOHMANN 117698
LARISSA EMY KASSUYA 102452

TRABALHO FINAL -
GERENCIAMENTO DE PROJETOS DE SOFTWARE

Trabalho apresentado ao curso de graduação de Informática da Universidade Estadual de Maringá, na disciplina de Gerenciamento de Projetos de Software.

MARINGÁ

2025

SUMÁRIO

1. PLANO DE GERENCIAMENTO DE PROJETO DE SOFTWARE..... 4
1.1. Introdução..... 4
1.1.1. Escopo do Projeto..... 4
Diagrama de Caso de Uso..... 4
Figura 1: Diagrama de Casos de Uso..... 4
1.1.2. Abordagem do Projeto..... 5
1.2. Análise de Riscos..... 5
Tabela 1: Identificação e Classificação dos Riscos..... 5
Tabela 2: Monitoramento dos riscos..... 6
1.3. Recursos Físicos..... 7
Tabela 3: Custos Diretos..... 7
Custos Indiretos..... 8
1.4. Estimativa de Custo e de Esforço..... 8
1.5. Gerenciamento e Cronograma de Projeto..... 11
Estrutura Analítica de Projeto (WBS)..... 11
Figura 2: Estrutura Analítica de Projeto (WBS)..... 11
Figura 3: Rede PERT..... 11
Figura 4: Diagrama de Gantt..... 12
1.6. Política de Gerenciamento do Trabalho..... 12
1.6.1. Gerenciamento de Configuração de Artefatos..... 12
1.6.2. Padrões para nomes de Branches..... 13
1.6.3. Criação de branches..... 13
Figura 5 : Git flow..... 14
1.6.4. Padrões de commit..... 14
1.6.5. Pull request..... 14
1.6.6. Revisão de código..... 15
1.7. Considerações Finais..... 15
1.8. Referências Bibliográficas..... 16
2. DOCUMENTAÇÃO DO SOFTWARE..... 16
2.1. Projeto de Banco de Dados..... 16
2.1.1. Visão Geral e Diagrama de Entidade-Relacionamento (DER)..... 16
Figura 6: Diagrama de Entidade-Relacionamento (DER) do banco de dados.... 17
2.1.2. Estrutura de Tabelas..... 17
Tabela 4: users..... 17
Tabela 5: scouts..... 18
Tabela 6: presence_lists..... 18
Tabela 7: presence_list_scouts (Tabela de Junção)..... 18
2.1.3. Relacionamentos..... 19
2.1.4. Chaves, Restrições e Índices..... 19
2.2. Link do Repositório do código-fonte..... 20

1. PLANO DE GERENCIAMENTO DE PROJETO DE SOFTWARE

1.1. Introdução

Este projeto teve seu início com base em uma matéria do curso bacharelado em informática da UEM, com foco em desenvolver um sistema para a comunidade. Foi escolhido o grupo dos escoteiros de Maringá para atender uma de suas demandas, um registro de presenças de forma mais automática, facilitando o registro de vários escoteiros em um menor período de tempo.

1.1.1. Escopo do Projeto

O projeto consiste no desenvolvimento de um site para registro de presença de escoteiros do Grupo Escoteiro Verde Canção em eventos, com o objetivo de simplificar e tornar mais seguro o controle de participações. Por meio de uma interface acessível e responsiva, os monitores poderão registrar a presença dos escoteiros escaneando um código QR Code ou registrar a presença de cada escoteiro manualmente. O sistema permitirá consultar e exportar a listagem atendendo as normas de proteção de dados e as boas práticas de usabilidade.

Diagrama de Caso de Uso

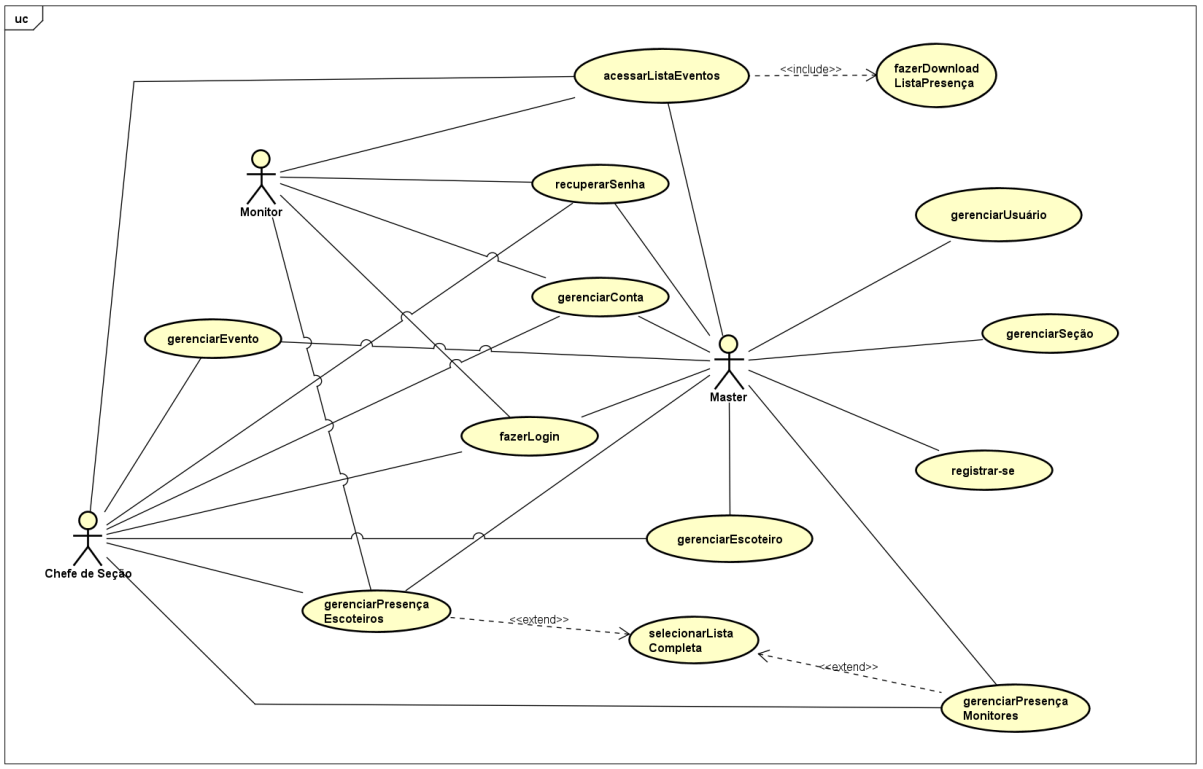

Figura 1: Diagrama de Casos de Uso.

1.1.2. Abordagem do Projeto

O projeto foi desenvolvido através de encontros periódicos realizados presencialmente e de maneira remota através de chamadas via Google Meet. Houve uma divisão de papéis, onde cada aluno foi designado a uma área de atuação, bem como o desenvolvimento colaborativo na elaboração do documento de várias etapas. Os papéis foram estabelecidos em:

- Gustavo Bonassoli → Back-end;
- Ingrid Lohmann → Front-end;
- Larissa Emy Kassuya → Design.

Foram utilizadas algumas ferramentas para a elaboração do projeto: Astah, Google Meet, GitHub, Trello, Visual Studio Code (VS Code) e Figma. Seus custos e descrições serão explicados posteriormente.

Além disso, foi realizado pela equipe dos escoteiros um teste A/B, no qual seria disponibilizado o registro através do QR Code e também de uma lista. O site foi disponibilizado ao cliente para que os monitores pudessem testar as duas funcionalidades e ao final, um questionário foi disponibilizado para que os participantes pudessem dar o seu feedback. Os mesmos foram orientados a realizar os testes através de um dispositivo móvel (celular) para que a simulação fosse feita da forma mais fiel possível.

No total foram 14 participantes e as respostas podem ser acessadas através do link. A preferência da grande maioria dos participantes foi pelo registro manual, pois esse método é independente do escoteiro estar portando ou não o seu QR Code.

1.2. Análise de Riscos

Esta seção apresenta a identificação e a avaliação dos principais riscos potenciais relacionados ao desenvolvimento e à operação do sistema de registro de presença para escoteiros. A análise visa reconhecer possíveis ameaças ao sucesso do projeto, classificá-las de acordo com sua probabilidade de ocorrência e impacto, e indicar as estratégias de monitoramento e mitigação a serem adotadas. Dessa maneira, busca-se aumentar a eficiência e a confiabilidade do sistema, reduzindo riscos e garantindo que as operações sejam conduzidas de maneira segura e eficaz.

Tabela 1: Identificação e Classificação dos Riscos

	Risco	Categoria	Probabilidade	Impacto
1	Problemas de leitura do QR Code	Operacional	Moderada	2
2	Conectividade instável	Técnico	Alta	3

	no local do evento			
3	Vazamento ou acesso não autorizado aos dados	Segurança/LGPD	Baixa	4
4	Lentidão ou instabilidade do sistemas sob alta demanda	Técnico/Operacional	Moderada	3
5	Resistência dos monitores para usar o sistema	Humano/Operacional	Moderada	2
6	Exportação de relatórios não funcionando corretamente	Operacional/Técnico	Baixa	2

Tabela 2: Monitoramento dos riscos

Risco	**Ação de monitoramento**	**Critérios de alerta**	**Ação de contingência**	**Responsável**
Conectividade instável no local do evento	Verificação simples do sinal antes do registro e acompanhamento durante o evento	Queda ou instabilidade da conexão por mais de 5 minutos	Ativar registro offline para sincronização posterior	Equipe de Suporte
Vazamento ou acesso não autorizado aos dados	Monitoramento de acessos e tentativas de login não-autorizadas	Qualquer acesso não-autorizado registrado no sistema de log	Isolar o sistemas, realizar backup e notificar os responsáveis	Equipe de TI/Segurança
Lentidão ou instabilidade do sistemas sob alta demanda	Monitoramento contínuo do tempo de resposta e disponibilidade do servidor	Tempo de resposta acima de 5s ou indisponibilidade do site	Ativação de infraestrutura de contingência e comunicação à equipe técnica para otimização e escalabilidade do servidor	Equipe de suporte/Desenvolvedores
Exportação de relatórios não funcionando corretamente	Verificação periódica da função de exportação e testes após atualizações	Exportação não concluída ou arquivo gerado com erro	Exportação manual dos registros direto do banco de dados e comunicação à equipe técnica para correção do problema	Equipe de Suporte / Desenvolvedores

A identificação e a análise dos riscos apresentados permitem uma compreensão clara dos principais pontos críticos do sistema e de seu impacto potencial no sucesso do projeto. A partir desse entendimento, torna-se possível adotar medidas preventivas e planos de contingência para mitigar ameaças e responder de maneira eficaz às situações adversas. Dessa forma, busca-se aumentar a eficiência, garantir a continuidade e melhorar a confiabilidade do sistema de registro de presença, alinhando-o às expectativas e às necessidades dos envolvidos.

1.3. Recursos Físicos

Os recursos físicos necessários para o desenvolvimento do projeto e sua implementação estão descritos na tabela abaixo.

Tabela 3: Custos Diretos

NOME	DESCRIÇÃO	CUSTO
Astah	Ferramenta de modelagem	Gratuito - Licença acadêmica para estudantes
Google Meet	Ferramenta de videoconferência	Gratuito
GitHub	Plataforma de hospedagem de código-fonte e arquivos	Gratuito
Trello	Ferramenta de gerenciamento de projeto	Gratuito
Visual Studio Code (VS Code)	Editor de código-fonte	Gratuito
Figma	Editor gráfico de vetor e prototipagem de projetos de design	Gratuito
Ferramentas utilizadas no desenvolvimento	Node.js React	Gratuito
Servidor	Vercel	Gratuito
Banco de Dados	MySQL	Gratuito
WBS	Miro	Gratuito

Pert	Miro	Gratuito
Gantt	Notion	Gratuito

Custos Indiretos

Num âmbito geral, houveram custos indiretos relacionados aos recursos físicos, tais como, transporte, energia elétrica e internet.

1.4. Estimativa de Custo e de Esforço

Complexidade de Atores

Tipo de ator	Complexidade	Pontos	Número de atores	Resultado
Humanos	Alta	3	3	9
Linhas de comando	Média	2	0	0
APIs	Baixa	1	0	0
				Total: 9

Complexidade de Casos de Uso

Complexidade do Caso de Uso	Pontos	Número de Casos de uso	Resultado
Simples	5	3	15
Médio	10	10	100
Complexo	15	0	0
			Total: 115

$$PCUNA = CA + CCU$$
$$PCUNA = 9 + 115$$
$$PCUNA = 124$$

Fatores Técnicos

Sigla	Fator	Peso	Influência	Resultado
T1	Sistema Distribuído	2	0	0
T2	Performance	2	1	2
T3	Eficiência de usuário final	1	1	1
T4	Complexidade de processamento	1	3	3
T5	Projeto visando código reusável	1	3	3
T6	Facilidade de instalação	0.5	1	0.5
T7	Facilidade uso	0.5	1	0.5
T8	Portabilidade	2	3	6
T9	Facilidade de mudança	1	2	2
T10	Concorrência	1	2	2
T11	Segurança	1	1	1
T12	Acesso fornecido a terceiros	1	0	0
T13	Necessidade de treinamento	1	2	2
				Total: 23

$$TCF = 0,6 + (0,01 * TFactor)$$
$$TCF = 0,6 + (0,01 * 23)$$
$$TCF = 0,83$$

Fatores Ambientais

Sigla	Fator	Peso	Influência	Resultado
E1	Familiaridade com o processo de desenvolvimento	1,5	4	6
E2	Experiência com a aplicação	0,5	4	2
E3	Experiência com a orientação a objetos	1	3	3
E4	Capacidade do analista líder	0,5	3	1,5
E5	Motivação	1	2	2
E6	Estabilidade de requisitos obtidos historicamente	2	3	6
E7	Equipe em tempo parcial	-1	0	0
E8	Dificuldade com a linguagem de programação	-1	3	-3
				Total: 17,5

$$EF = 1,4 - (0,03 * EFactor)$$
$$EF = 1,4 - (0,03 * 17,5)$$
$$EF = 0,875$$

$$PCUA = PCUNA * TCF * EF$$
$$PCUA = 124 * 0,83 * 0,875$$
$$PCUA = 90,055 \; ou \; 90 \; Use \; Case \; Points$$

$$E = PCUA * IP$$
$$E = 90,055 * 22$$
$$E = 1981,21 \; horas \; de \; trabalho$$

$$Custo = E * R\$50,00$$
$$Custo = 1981,21 * 50$$
$$Custo = R\$99060,50$$

1.5. Gerenciamento e Cronograma de Projeto

Estrutura Analítica de Projeto (WBS)

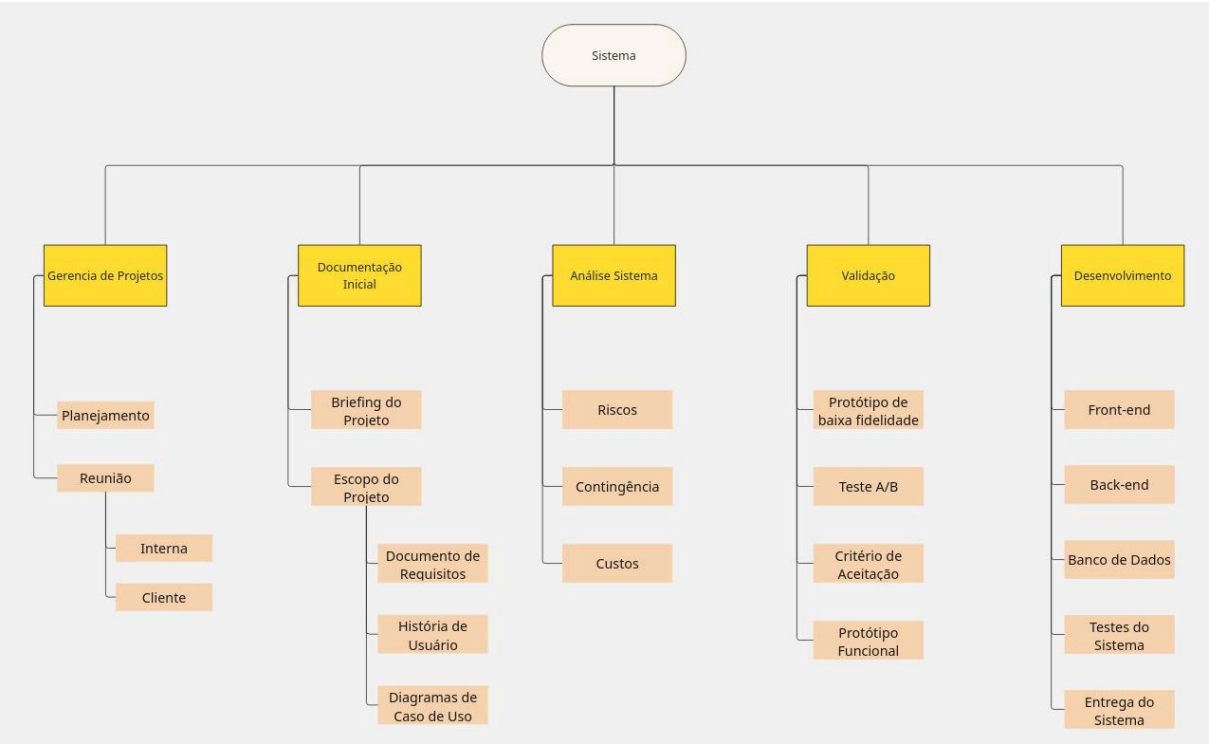

Figura 2: Estrutura Analítica de Projeto (WBS)

Rede PERT

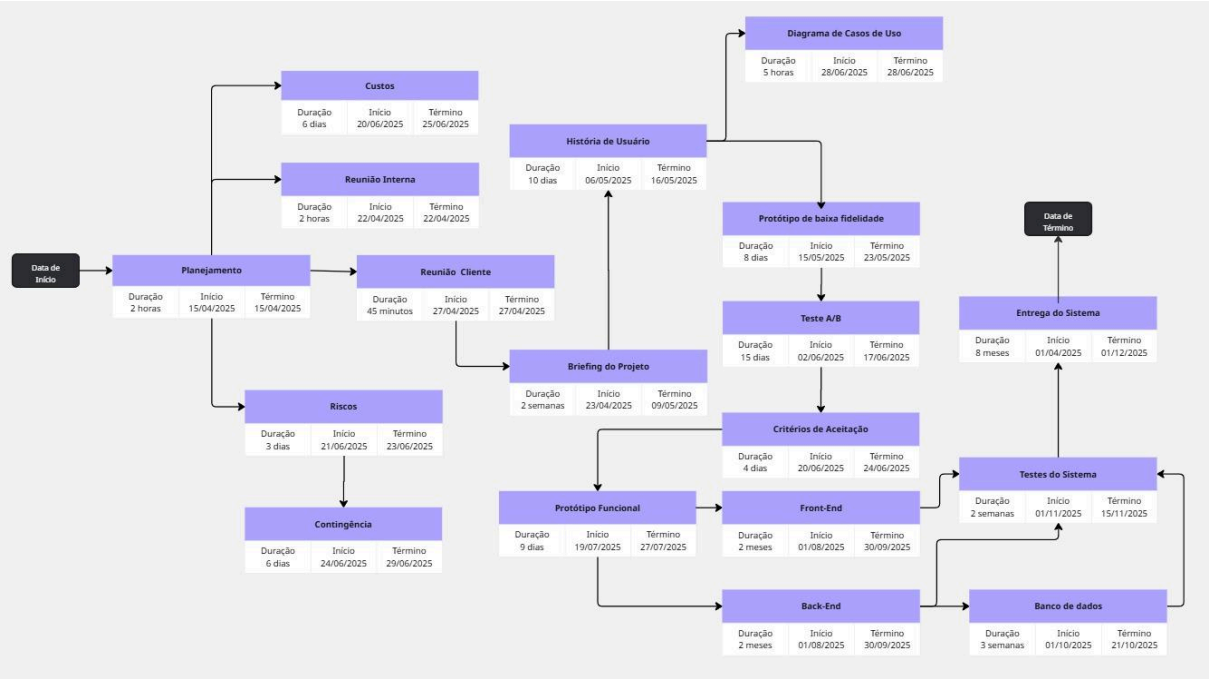

Figura 3: Rede PERT

Diagrama de Gantt

Nome da Tarefa	Duração	Status	Prioridade	Data de Início	Data de Término
Planejamento	2 horas	Done	Medium	15 de abril de 2025	15 de abril de 2025
Reunião Interna	2 horas	Done	Medium	22 de abril de 2025	22 de abril de 2025
Reunião Cliente	45 minutos	Done	Medium	28 de abril de 2025	28 de abril de 2025
Briefing do Projeto	2 semanas	Done	Medium	5 de maio de 2025	16 de maio de 2025
História de Usuário	10 dias	Done	Medium	6 de maio de 2025	16 de maio de 2025
Protótipo de baixa fidelidade	8 dias	Done	High	15 de maio de 2025	26 de maio de 2025
Teste A/B	15 dias	Done	High	2 de junho de 2025	20 de junho de 2025
Critérios de Aceitação	4 dias	Done	Medium	20 de junho de 2025	25 de junho de 2025
Custos	6 dias	Done	Low	20 de junho de 2025	27 de junho de 2025
Riscos	3 dias	Done	Medium	23 de junho de 2025	25 de junho de 2025
Contingência	6 dias	Done	Medium	24 de junho de 2025	1 de julho de 2025
Diagrama de Casos ABRIR	5 horas	Done	Low	30 de junho de 2025	30 de junho de 2025
Protótipo Funcional	9 dias	In progress	Medium	21 de julho de 2025	31 de julho de 2025
Front-End	2 meses	Not started	High	1 de agosto de 2025	30 de setembro de 2025
Back-End	2 meses	Not started	High	1 de agosto de 2025	30 de setembro de 2025
Banco de dados	3 semanas	Not started	High	1 de outubro de 2025	21 de outubro de 2025
Testes do Sistema	2 semanas	Not started	Medium	3 de novembro de 2025	14 de novembro de 2025
Entrega do Sistema	8 meses	Not started	Low	1 de dezembro de 2025	1 de dezembro de 2025

Figura 4: Diagrama de Gantt

1.6. Política de Gerenciamento do Trabalho

Esta seção apresenta os mecanismos técnicos usados pela equipe para organizar, controlar e rastrear todos os artefatos do projeto. Essas práticas garantem eficiência, facilitam a comunicação e asseguram a qualidade e consistência dos entregáveis, sendo detalhadas nos tópicos a seguir.

1.6.1. Gerenciamento de Configuração de Artefatos

Git:

Utilizado para realizar o controle de versão do código-fonte e demais artefatos do projeto. Através do Git, todas as alterações são registradas com histórico, o que garante rastreabilidade, recuperação de versões anteriores e facilidade de integração entre diferentes membros da equipe.

GitHub:

Usado como repositório remoto para armazenar e centralizar o código e demais artefatos do projeto, facilitando o trabalho colaborativo e a visibilidade do histórico de mudanças. Por meio do GitHub, a equipe realiza a integração contínua,

revisões de código (pull requests) e acompanha o status das demandas e atividades associadas ao desenvolvimento.

1.6.2. Padrões para nomes de Branches

A branch deve deixar claro qual é o CARD do Trello que ela está vinculada, bem como o ambiente (dev, hmg ou prod) para qual ela é destinada, seguindo o seguinte esquema: **AMBIENTE-CARD-123**.

Exemplos:
DEV-FRONT-1142: Branch do card FRONT-1142 para o ambiente de DEV.
HMG-FRONT-123: Branch do card FRONT-123 para o ambiente de HMG.
FRONT-1182: Branch do card FRONT-1182 para o ambiente de PROD (master).

No ambiente de PROD deve-se omitir o nome do ambiente no nome da branch.

1.6.3. Criação de branches

As branches base são:

- **dev**: ambiente de desenvolvimento
- **hmg**: ambiente de homologação
- **master**: ambiente de produção

As branches de trabalho devem ser criadas a partir da branch de destino, por exemplo, usando o card FRONT-123 para ser entregue:

Criar a branch DEV-FRONT-123 a partir da dev
Criar a branch HMG-FRONT-123 a partir da hmg
Criar a branch FRONT-123 a partir da master

Abaixo está representado o git flow do projeto, onde as branches features terão as estruturas citadas acima.

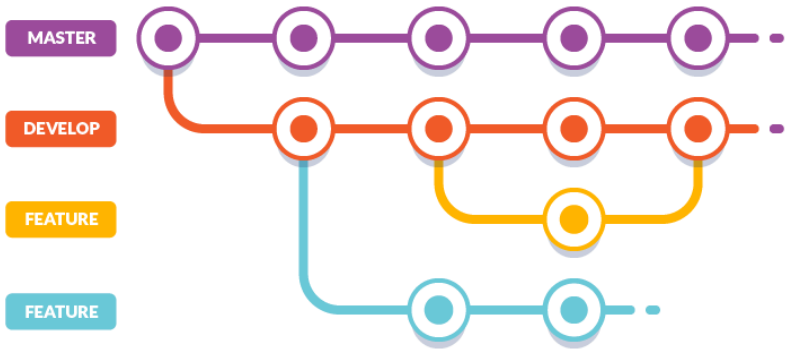

Figura 5 : Git flow

1.6.4. Padrões de commit

O tipo deve ser um dos seguintes:

docs: Alterações exclusivas de documentação
feat: Adição de novas features
fix: Correções de bugs
refactor: Alterações que não corrigem bugs nem adicionam features
revert: Commit de reversão
style: Alterações de estilos apenas
test: Alterações para adicionar testes novos ou corrigir existentes

O resumo deve prover uma descrição sucinta das alterações contidas no commit. Deve-se utilizar o verbo no presente do indicativo utilizando a terceira pessoa do singular: Refatora, adiciona, atualiza, remove, etc. Além disso, o resumo deve ser escrito em letras minúsculas e sem ponto final.

Exemplo:

```
feat: adiciona tela de login
```

1.6.5. Pull request

No título deve ser adicionado o código do card (tarefa do Trello).
Exemplo: FRONT-26: cria componente básico de input

Deve ser adicionado a descrição do que adicionado, atualizado ou removido no pull request, seguindo o template que será disponibilizado.

1.6.6. Revisão de código

A revisão do Pull Request deve ser feita com atenção ao código apresentado, adicionando comentários para esclarecer dúvidas, sugerir melhorias ou indicar correções obrigatórias, registrando uma tarefa quando necessário. Deve-se, em seguida, avaliar a qualidade do código sob a perspectiva de sua eficiência e conformidade para entrar em produção. Por fim, estando o resultado satisfatório, o Pull Request deve ser aprovado para permitir que o autor dê continuidade à integração por meio do merge.

1.7. Considerações Finais

Ao final deste projeto, conclui-se que a aplicação sistemática de práticas de gerenciamento de software foi um fator crítico para o sucesso no desenvolvimento da solução de registro de presença. A jornada, que iniciou com o levantamento de requisitos junto ao Grupo Escoteiro Verde Canção, foi estruturada por meio de um plano robusto, abrangendo desde a análise de riscos, como a conectividade instável, até a definição de um fluxo de trabalho colaborativo com o Git e o GitHub.

A utilização de ferramentas como a WBS e a Rede PERT permitiu uma visualização clara das dependências e do cronograma, enquanto a política de gerenciamento de configuração garantiu a integridade e a qualidade do código-fonte. Um dos principais aprendizados foi a validação da importância do feedback do usuário final, evidenciado pelo teste A/B realizado com um protótipo funcional, que direcionou a funcionalidade para uma abordagem de lista manual, mais alinhada às necessidades reais dos monitores.

Portanto, este trabalho não apenas resultou em um produto de software que atende a uma necessidade específica da comunidade — materializado em um protótipo funcional acessível publicamente em www.escoteiroverdecancao.com.br ou https://frontend-rb.vercel.app/login e cujo desenvolvimento está documentado nos repositórios do projeto —, mas também serviu como uma valiosa experiência prática. Fica demonstrado que um gerenciamento de projeto bem executado é a base que sustenta a qualidade técnica, a previsibilidade e o alinhamento do produto final com as expectativas do cliente.

1.8. Referências Bibliográficas

NESTJS. NestJS Documentation. [S. l.], 2024. Disponível em: https://docs.nestjs.com/. Acesso em: 1 jul. 2025.

META PLATFORMS. React Documentation. [S. l.], 2024. Disponível em: https://react.dev/. Acesso em: 1 jul. 2025.

VITE. Vite Documentation. [S. l.], 2024. Disponível em: https://vitejs.dev/. Acesso em: 1 jul. 2025.

HUNT, Andrew; THOMAS, David. O Programador Pragmático: De Aprendiz a Mestre. Porto Alegre: Bookman, 2010.

MARTIN, Robert C. Código Limpo: Habilidades Práticas do Agile Software. Rio de Janeiro: Alta Books, 2009.

2. DOCUMENTAÇÃO DO SOFTWARE

2.1. Projeto de Banco de Dados

Essa seção descreve a modelagem do banco de dados relacional projetado para o sistema de gerenciamento de listas de presença de escoteiros. A estrutura foi implementada utilizando o ORM (Object-Relational Mapping) TypeORM sobre um banco de dados SQLite, mas os conceitos de modelagem são aplicáveis a qualquer sistema de banco de dados relacional como PostgreSQL ou MySQL.

2.1.1. Visão Geral e Diagrama de Entidade-Relacionamento (DER)

O modelo é composto por três entidades principais: **Usuários** (*users*), **Escoteiros** (*scouts*) e **Listas de Presença** (*presence_lists*). A relação central do sistema é a de **Muitos-para-Muitos** entre escoteiros e listas de presença, que é resolvida através de uma tabela de junção.

Abaixo, um diagrama conceitual que representa a estrutura:

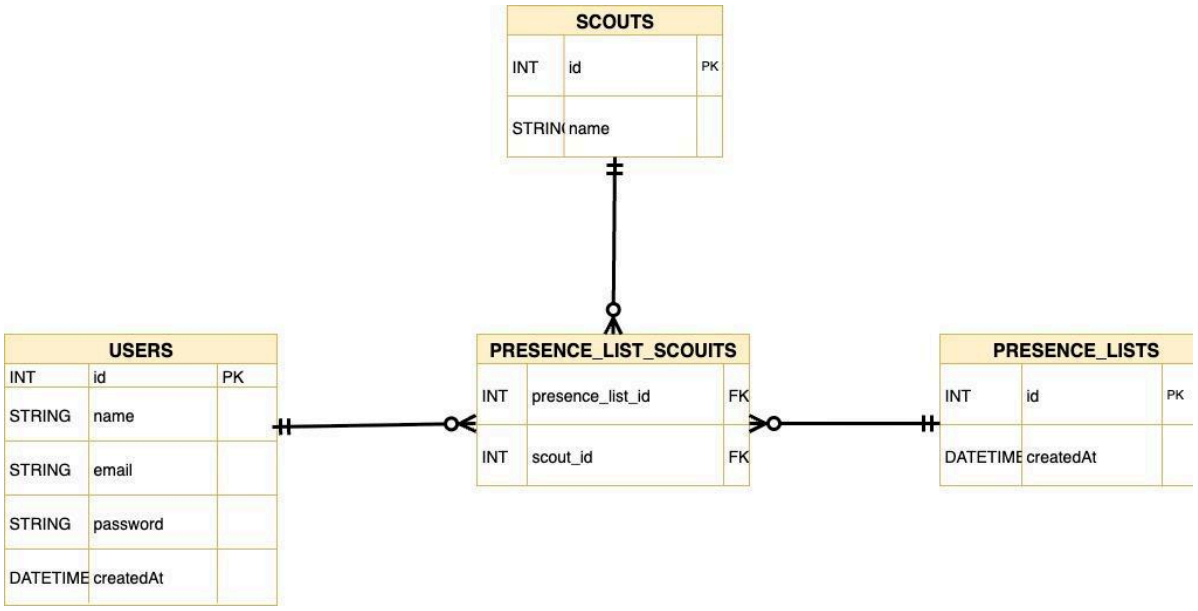

Figura 6: Diagrama de Entidade-Relacionamento (DER) do banco de dados.

2.1.2. Estrutura de Tabelas

Tabela 4: users

Armazena as informações dos usuários que podem acessar o sistema (ex: chefes escoteiros).

Coluna	Tipo de Dado	Chaves/Restrições	Descrição
id	INTEGER	**Chave Primária (PK)**, Auto-incremento	Identificador numérico único para cada usuário.
name	VARCHAR	NOT NULL	Nome completo do usuário.
email	VARCHAR	NOT NULL, **UNIQUE**	Endereço de e-mail do usuário, usado para login. A restrição UNIQUE impede e-mails duplicados.
password	VARCHAR	NOT NULL	Senha do usuário, armazenada de forma criptografada (hash).

createdAt	DATETIME	NOT NULL	Data e hora exatas em que o registro do usuário foi criado.

Tabela 5: scouts

Armazena o cadastro de todos os escoteiros.

Coluna	Tipo de Dado	Chaves/Restrições	Descrição
id	INTEGER	**Chave Primária (PK)**	Identificador numérico único para cada escoteiro. Este ID é fornecido pelo usuário no momento do cadastro.
name	VARCHAR	NOT NULL	Nome completo do escoteiro.

Tabela 6: presence_lists

Armazena um registro para cada lista de presença que é salva.

Coluna	Tipo de Dado	Chaves/Restrições	Descrição
id	INTEGER	**Chave Primária (PK)**, Auto-incremento	Identificador numérico único para cada lista de presença.
createdAt	DATETIME	NOT NULL	Data e hora exatas em que a lista de presença foi confirmada e salva.

Tabela 7: presence_list_scouts (Tabela de Junção)

Esta tabela é criada automaticamente pelo TypeORM para resolver o relacionamento de Muitos-para-Muitos. Ela não possui uma entidade de código correspondente, mas é fundamental para a estrutura do banco.

Coluna	Tipo de Dado	Chaves/Restrições	Descrição
presence_list_id	INTEGER	**Chave Primária**	Referencia o ID da

		Composta, **Chave Estrangeira (FK)** para presence_lists(id)	lista de presença.
scout_id	INTEGER	**Chave Primária Composta**, **Chave Estrangeira (FK)** para scouts(id)	Referencia o ID do escoteiro presente na lista.

2.1.3. Relacionamentos

O relacionamento principal do sistema é entre scouts e *presence_lists*.

- **Tipo:** Muitos-para-Muitos (N:M)
- **Descrição:**
  - Um escoteiro (*scout*) pode estar presente em várias listas de presença (*presence_lists*).
  - Uma lista de presença (presence_list) pode conter vários escoteiros (scouts).
- **Implementação:** Este relacionamento é implementado através da tabela de junção *presence_list_scouts*. Cada linha nesta tabela representa a presença de um escoteiro específico em uma lista de presença específica, conectando um registro da tabela scouts a um registro da tabela *presence_lists*.

2.1.4. Chaves, Restrições e Índices

- **Chaves Primárias (Primary Keys):**
  - **users(id):** Garante que cada usuário seja único.
  - **scouts(id):** Garante que cada escoteiro seja único.
  - **presence_lists(id):** Garante que cada lista de presença seja única.
  - **presence_list_scouts(presence_list_id, scout_id):** A chave composta garante que um mesmo escoteiro não possa ser adicionado duas vezes na mesma lista.
- **Chaves Estrangeiras (Foreign Keys):**
  - **presence_list_scouts(presence_list_id) -> presence_lists(id):** Garante a integridade referencial. Não é possível registrar uma presença em uma lista que não existe.
  - **presence_list_scouts(scout_id) -> scouts(id):** Garante a integridade referencial. Não é possível registrar a presença de um escoteiro que não está cadastrado.
- **Restrições (Constraints):**
  - **UNIQUE:** A restrição na coluna users(email) é crucial para o sistema de autenticação, impedindo o cadastro de múltiplos usuários com o mesmo e-mail.

    - **NOT NULL:** Aplicada na maioria das colunas para garantir que os registros não sejam salvos com dados essenciais faltando.
- **Índices (Indexes):**
    - **Automáticos:** Todas as chaves primárias (id em cada tabela principal) e colunas com a restrição UNIQUE (email na tabela users) são automaticamente indexadas pelo sistema de banco de dados. Isso otimiza drasticamente a velocidade das operações de busca (ex: encontrar um usuário pelo ID ou e-mail).
    - **Implícitos:** As chaves estrangeiras também são geralmente indexadas para acelerar as operações de JOIN entre as tabelas, o que é fundamental para consultar quais escoteiros estavam em uma determinada lista.

2.2. Link do Repositório do código-fonte

A gestão de configuração de software do projeto foi centralizada na plataforma GitHub, utilizando o sistema de controle de versão Git. Adotou-se uma abordagem de separação de responsabilidades, dividindo o projeto em três repositórios independentes para facilitar o desenvolvimento, a manutenção e o deploy.

Os artefatos do projeto estão organizados da seguinte forma:

- Repositório do Frontend: Contém todo o código-fonte da aplicação cliente, desenvolvida em React, responsável pela interface e interação com o usuário.
  URL: https://github.com/UndefinedUEM/frontend-rb

- Repositório do Backend: Abrange o código-fonte da aplicação servidora, desenvolvida em Node.js com NestJS, que gerencia a lógica de negócio, o acesso ao banco de dados e a API.
  URL: https://github.com/UndefinedUEM/backend-rb

- Repositório de Documentação: Centraliza todos os documentos pertinentes ao projeto, como a modelagem do banco de dados, especificações e o relatório de gerenciamento.
  URL: https://github.com/UndefinedUEM/documentos